

# The C Programming Language

Nathan Warner



Northern Illinois  
University

Computer Science  
Northern Illinois University  
United States

## Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                               | <b>2</b>  |
| <b>2</b> | <b>Input / output</b>                             | <b>3</b>  |
| 2.1      | Working with files . . . . .                      | 8         |
| 2.2      | Binary I/O . . . . .                              | 14        |
| 2.3      | I/O helpers . . . . .                             | 15        |
| 2.3.1    | Flushing . . . . .                                | 15        |
| 2.3.2    | File positioning . . . . .                        | 16        |
| 2.3.3    | EOF and Errors . . . . .                          | 18        |
| 2.3.4    | Removing and renaming files . . . . .             | 19        |
| <b>3</b> | <b>Strings</b>                                    | <b>20</b> |
| <b>4</b> | <b>Memory operations in string.h</b>              | <b>25</b> |
| <b>5</b> | <b>stdlib.h</b>                                   | <b>27</b> |
| 5.1      | Memory management . . . . .                       | 27        |
| 5.2      | String to number conversions . . . . .            | 29        |
| 5.3      | Program termination and process control . . . . . | 31        |
| 5.4      | Random numbers . . . . .                          | 33        |
| 5.5      | Sorting and searching . . . . .                   | 34        |
| 5.6      | Integer math helpers . . . . .                    | 36        |
| <b>6</b> | <b>Concepts</b>                                   | <b>37</b> |
| 6.1      | C99 . . . . .                                     | 40        |

## Introduction

- **C++ features not in C:** This is clearly a non-exhaustive list, just the less obvious ones:
  1. Classes
  2. Member functions
  3. Constructors
  4. Destructors
  5. Access control: public, private, protected
  6. Inheritance
  7. Operator overloading
  8. Function overloading
  9. constexpr functions
  10. constexpr
  11. constexpr
  12. Namespaces
  13. References
  14. Move semantics
  15. RAII as a language-supported idiom
  16. Exceptions
  17. try / catch / throw
  18. new / delete
  19. Iterators
  20. Lambdas
  21. Capturing lambdas
  22. auto type deduction in the C++ sense
  23. decltype
  24. typeid
  25. dynamic\_cast, static\_cast, reinterpret\_cast, const\_cast
  26. Default function arguments
  27. Default member initializers
  28. Uniform initialization: initialization
  29. Member initializer lists
  30. this pointer
  31. Scoped enums: enum class
  32. bool as a built-in fundamental type in older C comparison
  33. References to functions/objects
  34. Inline variables
  35. Overload resolution
  36. Member pointers

## Input / output

- **stdio.h:** In C, input/output is mostly handled through the standard I/O library, declared in:

```
0  #include <stdio.h>
```

- **Standard streams:** When a C program starts, it automatically has three standard streams:

```
0  stdin
1  stdout
2  stderr
```

- **printf:** Printf prints formatted text to stdout.

```
0  printf("%s %s", "Hello, ", "world!");
```

The placeholders are called format specifiers. Common ones include

```
0  %d      int
1  %u      unsigned int
2  %ld     long
3  %lu     unsigned long
4  %lld    long long
5  %llu    unsigned long long
6
7  %f      double
8  %lf     double in scanf, but also accepted in printf
9  %c      char
10 %s      string / char array
11 %p      pointer address
12 %%     literal percent sign
```

For %p, cast the pointer to void\*. Note that the language itself does not make printf typesafe. Wrong format specifiers cause **undefined behavior**.

- **fprintf:** The fprintf function in C is used to write formatted data directly to an output stream, which is most commonly a file. It works identically to printf, except it requires a destination file pointer as its very first argument.

```
0  int fprintf(FILE *stream, const char *format, ...);
```

- **sprintf:** The sprintf function in C formats and stores a series of characters and values into a string buffer instead of printing them directly to the console. It is defined in the <stdio.h> standard library.

```
0  int sprintf(char *str, const char *format, ...);
```

Returns the total number of characters successfully written, excluding the null terminator (`\0`). If an error occurs, it returns a negative value.

- **snprintf**:

```
0  int snprintf ( char * s, size_t n, const char * format, ...  
    ↪ );
```

Composes a string with the same text that would be printed if `format` was used on `printf`, but instead of being printed, the content is stored as a C string in the buffer pointed by `s` (taking `n` as the maximum buffer capacity to fill).

If the resulting string would be longer than `n-1` characters, the remaining characters are discarded and not stored, but counted for the value returned by the function.

A terminating null character is automatically appended after the content written.

Returns the number of characters that would have been written if `n` had been sufficiently large, not counting the terminating null character. If an encoding error occurs, a negative number is returned.

The key rule is **size** is the total buffer size, including space for the null terminator.

```
0  char buf[10];  
1  snprintf(buf, sizeof buf, "hello");
```

`sizeof buf` is 10, so `snprintf` may write at most 9 visible characters plus the final `\0`.

```
0  char buf[6];  
1  snprintf(buf, sizeof buf, "hello");
```

This fits exactly. So, this is safe.

```
0  char buf[5];  
1  
2  int n = snprintf(buf, sizeof buf, "hello");
```

`snprintf` writes

```
0  'h' 'e' 'l' 'l' '\0'
```

in this case the return value is still 5, because "hello" would have required 5 visible characters, not counting the null terminator. `snprintf` returns the number of characters that would have been written, excluding the null terminator.

```
0  char buf[5];
1
2  int n = snprintf(buf, sizeof buf, "hello");
3
4  if (n < 0) {
5      // encoding or formatting error
6  } else if ((size_t)n >= sizeof buf) {
7      // output was truncated
8  } else {
9      // output fit
10 }
```

You can call `snprintf` once to find out how much space is needed:

```
0  int n = snprintf(NULL, 0, "value = %d", 123);
```

- **scanf**: `scanf` reads formatted input from `stdin`.

```
0  int x;
1  scanf("%d", &x)
```

`Scanf` needs the address because it has to modify the variable. `Scanf` returns the number of successful conversions. You should usually check the return value

```
0  int x;
1  if (scanf("%d", &x) != 1) {
2      fprintf(stderr, "Invalid integer\n");
3  }
```

- **puts**: Writes a string to the console

```
0  puts("Hello, world!");
```

- **getchar**: reads one character from `stdin`

```
0  int c;
1  c = getchar();
```

Notice that `getchar` returns `int`, not `char`. That is because it may return EOF, which is a special value indicating end-of-file or error.

```

0  int c;
1  while ((c=getchar()) != EOF) putchar(c);

```

- **putchar**: Writes one character to stdout

```

0  putchar('A');

```

- **fgets for line input**: Avoid

```

0  char name[32];
1  scanf("%s", name);

```

If the user types more than 31 characters, you can overflow the buffer. A safer approach is fgets:

```

0  char name[32];
1
2  printf("Enter your name: ");
3
4  if (fgets(name, sizeof name, stdin) == NULL) {
5      fprintf(stderr, "Input error\n");
6      return 1;
7  }
8
9  printf("Hello, %s", name);

```

Fgets has signature

```

0  char* fgets( char* str, int count, FILE* stream );

```

and returns **str** on success, null pointer on failure.

- **sscanf**: The sscanf function in C extracts and parses formatted data directly from a character string rather than reading from standard keyboard input (stdin). It is defined in the <stdio.h> library and stands for "String Scan Formatted".

```

0  int sscanf(const char *str, const char *format, ...);

```

```

0  const char *info = "ID:4023 Alice 28";
1  int id;
2  char name[20];
3  int age;
4
5  // Parse data by explicitly matching the literal "ID:" prefix
6  int items = sscanf(info, "ID:%d %19s %d", &id, name, &age);

```

- **%[set]:** A scanset. For instance, %[A-Za-z] reads only alphabetic characters.
- **%[^set]:** An inverted scanset. For instance, %[^,] reads everything up until it hits a comma.
- **\* Modifier:** Suppresses assignment. For example, %\*d matches and skips an entire integer without storing it.



## 2.1 Working with files

- **Header:** File I/O is mainly done through the same header

```
0 #include <stdio.h>
```

- **FILE type:** The central type is

```
0 FILE*
```

A FILE\* is a handle to an open file stream. You do not manipulate the file directly. You open it, get a FILE \*, use functions such as fgetc, fgets, fprintf, fread, and finally close it with fclose.

- **fopen:**

```
0 FILE *fopen(const char *restrict filename, const char
  ↪ *restrict mode);
```

Upon successful completion fopen(), fdopen(), and freopen() return a FILE pointer. Otherwise, NULL is returned and errno is set to indicate the error.

The mode string tells C whether you want to read, write, append, or work in binary mode.

| Mode | Meaning                                             |
|------|-----------------------------------------------------|
| "r"  | Open existing file for reading                      |
| "w"  | Open file for writing; destroys old contents        |
| "a"  | Open file for appending                             |
| "r+" | Open existing file for reading and writing          |
| "w+" | Open for reading and writing; destroys old contents |
| "a+" | Open for reading and appending                      |
| "rb" | Read binary                                         |
| "wb" | Write binary                                        |
| "ab" | Append binary                                       |

The primary difference between r+ and a+ modes in C file handling lies in how they handle file creation and where the data is written. While both allow you to read and write without erasing (truncating) the existing data, they behave very differently under the hood.

| Feature                     | r+ (Read / Update)                  | a+ (Append / Update)                  |
|-----------------------------|-------------------------------------|---------------------------------------|
| File Must Exist?            | Yes (returns NULL if missing)       | No (creates a new file if missing)    |
| Initial Pointer Position    | Beginning of the file               | End of the file                       |
| Overwrites Content?         | Yes (overwrites at current cursor)  | No (always forces writes to EOF)      |
| Effect of fseek() on Writes | Moves the cursor for the next write | Ignored (writes always go to the end) |

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     fclose(fp);
11     return 0;
12 }

```

- **fclose:**

```

0  int fclose(FILE* stream);

```

fclose closes an open file stream. It also flushes any buffered output before closing the file.

You should call fclose on every file you successfully opened with fopen.

Upon successful completion, 0 is returned. Otherwise, EOF is returned and errno is set to indicate the error.

EOF is a negative integer constant defined in <stdio.h>.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("output.txt", "w");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     fprintf(fp, "Hello file\n");
11
12     if (fclose(fp) != EOF) {
13         perror("fclose");
14         return 1;
15     }
16
17     return 0;
18 }

```

- **The restrict keyword:** restrict is a type qualifier used on pointers to promise the compiler that the pointer is the only means to access its underlying memory object

When you qualify a pointer with restrict, you guarantee that for the lifetime of that pointer, no other pointer or variable inside the same scope will read or write to that specific chunk of memory.

- **fprintf:**

```

0  int fprintf(FILE *restrict stream, const char *restrict
   ↪  format, ...);

```

fprintf works like printf, but it writes to a file stream instead of standard output.

Returns the number of characters written on success.

Returns a negative value on failure.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("numbers.txt", "w");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     int x = 10;
11     double y = 3.14;
12
13     fprintf(fp, "x = %d\n", x);
14     fprintf(fp, "y = %.2f\n", y);
15
16     fclose(fp);
17     return 0;
18 }

```

- **fputs:**

```

0  int fputs(const char *restrict s, FILE *restrict stream);

```

fputs writes a string to a file. Unlike puts, it does not automatically add a newline.

Returns a non-negative value on success. Returns EOF on failure.

- **fputc**

```

0  int fputc(int c, FILE *stream);

```

fputc writes a single character to a file. The character is passed as an int, but it usually represents a char.

Returns the character written on success. Returns EOF on failure.

- **fgetc:**

```

0  int fgetc(FILE *stream);

```

fgetc reads one character from a file.

It returns an int, not a char, because it needs to represent every possible character plus the special value EOF.

Returns the character read on success. Returns EOF if the end of the file is reached or if an error occurs.

```
0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     int ch;
11
12     while ((ch = fgetc(fp)) != EOF) {
13         putchar(ch);
14     }
15
16     fclose(fp);
17     return 0;
18 }
```

- **fgets:**

```
0  char *fgets(char *restrict s, int n, FILE *restrict stream);
```

fgets reads a line, or part of a line, from a file into a character buffer.

It reads at most  $n - 1$  characters and then adds the null terminator `'\0'`.

If it sees a newline before the limit, it includes the newline in the buffer.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     char line[256];
11
12     while (fgets(line, sizeof line, fp) != NULL) {
13         printf("%s", line);
14     }
15
16     fclose(fp);
17     return 0;
18 }

```

This is the normal way to read a text file line by line in C.

- **fscanf:**

```

0  int fscanf(FILE *restrict stream, const char *restrict
    ↪  format, ...);

```

fscanf reads formatted input from a file, similar to how scanf reads from standard input.

It is useful for simple structured input, such as integers, floats, and words.

However, fscanf can be fragile for complicated input. For many real programs, reading a full line with fgets and then parsing it manually is safer.

Returns the number of input items successfully matched and assigned.

Returns EOF if the end of the file is reached before any conversion happens, or if a read error occurs.

- **fputs**

## 2.2 Binary I/O

- **fwrite**

```
o  size_t fwrite(const void *restrict data_ptr, size_t  
    ↪ element_size, size_t n_elements, FILE *restrict stream);
```

Writes raw binary data from memory into a file. To write one `int`:

```
o  fwrite(&x, sizeof x, 1, fp);
```

To write an array:

```
o  fwrite(arr, sizeof arr[0], count, fp)
```

Returns the number of characters written on success.

Returns a negative value on failure.

- **fread:**

```
o  size_t fread(void ptr[restrict size * n], size_t size,  
    ↪ size_t n, FILE *restrict stream);
```

The function `fread()` reads `n` items of data, each `size` bytes long, from the stream pointed to by `stream`, storing them at the location given by `ptr`.

On success, `fread()` and `fwrite()` return the number of items read or written. This number equals the number of bytes transferred only when `size` is 1. If an error occurs, or the end of the file is reached, the return value is a short item count (or zero).

## 2.3 I/O helpers

### 2.3.1 Flushing

- `fflush`

```
0  int fflush(FILE* stream);
```

C file output is often buffered. That means data may sit in memory temporarily before being physically written to the file. `fflush` forces buffered output to be written immediately. It is mainly used for output streams.

Returns 0 on success. Returns EOF on failure.



### 2.3.2 File positioning

- **Intro:** Every open file stream has a current position. Reading or writing usually advances that position.

You can move around in a file using `fseek`, `ftell`, and `rewind`.

- **fseek:**

```
0  int fseek(FILE* stream, long offset, int whence);
```

`fseek` moves file position. Whence controls where the offset is measured from:

- **SEEK\_SET:** Beginning of file
- **SEEK\_CUR:** Current position
- **SEEK\_END:** End of file

Returns zero on success, nonzero on failure.

```
0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     if (fseek(fp, 0, SEEK_END) != 0) {
11         perror("fseek");
12         fclose(fp);
13         return 1;
14     }
15
16     fclose(fp);
17     return 0;
18 }
```

Moves position to EOF

- **ftell:**

```
0  long ftell(FILE* stream);
```

returns the current file position. For binary files, this is commonly used as a byte offset.

Returns current position on success, `-1L` on failure.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     fseek(fp, 0, SEEK_END);
11
12     long size = ftell(fp);
13
14     if (size == -1L) {
15         perror("ftell");
16         fclose(fp);
17         return 1;
18     }
19
20     printf("File size: %ld bytes\n", size);
21
22     fclose(fp);
23     return 0;
24 }

```

- **rewind:**

```

0  void rewind(FILE* stream);

```

reset the file position indicator in a stream

### 2.3.3 EOF and Errors

- **feof:**

```
0  int feof(FILE *stream);
```

The feof() function shall test the end-of-file indicator for the stream pointed to by stream.

The feof() function shall not change the setting of errno if stream is valid.

The feof() function shall return non-zero if and only if the end-of-file indicator is set for stream, otherwise returns zero.

- **ferror**

```
0  int ferror(FILE* stream);
```

ferror checks whether an error occurred on a file stream.

This is useful because some functions, such as fgetc, return EOF both for real end-of-file and for errors.

Returns nonzero if the error indicator is set, zero otherwise.

- **clearerr**

```
0  void clearerr(FILE* stream);
```

clearerr clears both the error indicator and the EOF indicator for a stream.

You usually do not need it in simple programs, but it matters if you want to continue using a stream after EOF or after handling an error.

- **perror:**

```
0  void perror(const char* s);
```

perror prints an error message describing the most recent library/system error.

It is commonly used after functions such as fopen, fread, fwrite, fflush, or fclose fail.

### 2.3.4 Removing and renaming files

- **remove:**

```
0  int remove(const char* filename);
```

Remove deletes a file.

Returns zero on success, nonzero on failure.

- **Rename**

```
0  int rename(const char* old, const char* new);
```

rename changes the name or path of a file.

It can also be used to move a file within the same filesystem.

Returns 0 on success. Returns nonzero on failure.

## Strings

- **String literals:** Consider

```
0 char* s1 = "Hello, world!";
1 const char* s2 = "Hello, world!";
```

These create read only string literals. You should always make string literals const. Attempting to modify a non-const string literal will cause a crash.

To make them mutable, attach them to a char array

```
0 char arr[] = "Hello, world!";
```

The compiler allocates a fresh block of writable memory on the stack (or global scope) and copies the contents of "Hello" into it. This memory is completely safe to change.

- **string.h:** The main C header for working with strings. Contains the standard C functions for working with null-terminated strings and raw memory blocks.
- **strlen**

```
0 size_t strlen(const char* s);
```

Returns the length of a C string, not counting the null terminator.

```
0 char s[] = "Hello";
1
2 sizeof s // 6, includes \0
3 strlen(s) // 5, excludes \0
```

- **strcmp:**

```
0 int strcmp(const char *s1, const char *s2);
```

Compares two strings lexicographically.

A negative return value implies s1 is less than s2. A return value of zero implies s1 equals s2. A positive return value implies s1 is greater than s2.

**Note:** Strings **cannot** be compared with ==. Using this operator simply compares addresses.

Because `strcmp` does not accept a maximum length argument, it evaluates characters sequentially until it detects a null-terminator (`\0`). If a maliciously crafted or malformed string lacks a null-terminator,

- Keep reading past the assigned memory buffer.
- Trigger undefined program behavior.
- Cause a segmentation fault (SIGSEGV) or application crash if it tries to read protected memory spaces.
- Potentially read adjacent data from the stack or heap, giving attackers a mechanism to bleed sensitive information

- **strncmp:**

```
0  int strncmp(const char* s1, const char* s2, size_t n);
```

Compares at most the first  $n$  characters of two strings.

Has same return behavior as `strcmp`.

- **strcpy:**

```
0  char* strcpy(char* dest, const char* src);
```

Copies the string `src` into `dest`, including the null terminator.

Returns `dest`.

- **strncpy**

```
0  char *strncpy(char *dest, const char *src, size_t n);
```

Copies at most  $n$  characters from `src` to `dest`.

Returns `dest`.

This function is **not** simply a safer `strcpy`. If `src` has length greater than or equal to  $n$ , `strncpy` does **not** append a null-terminator.

```

0  #include <stdio.h>
1  #include <string.h>
2
3  int main(void) {
4      char dest[6];
5
6      strncpy(dest, "hello world", sizeof dest - 1);
7      dest[sizeof dest - 1] = '\0';
8
9      printf("%s\n", dest);
10
11     return 0;
12 }

```

- **strcat:**

```

0  char *strcat(char *dest, const char *src);

```

Appends *src* to the end of *dest*. Starts at the null-terminator of *dest*. *Dest* must have a null-terminator and enough room for the result.

Returns *dest*.

- **strncat:**

```

0  char *strncat(char *dest, const char *src, size_t n);

```

Appends at most *n* characters from *src* to *dest*, then appends a null-terminator.

Returns *dest*

- **strchr:**

```

0  char* strchr(const char* s, int c);

```

Finds the first occurrence of character *c* in string *s*.

Returns pointer to first occurrence of character *c* in string *s*, or NULL if not found.

- **strrchr**

```

0  char* strrchr(const char* s, int c);

```

Finds the last occurrence of character *c* in string *s*.

Same return value as strchr

- **strstr**

```
0 char* strstr(const char* haystack, const char* needle);
```

Finds the first occurrence of string **needle** inside string **haystack**.

Returns pointer on success, NULL if not found.

- **strtok:**

```
0 char* strtok(char* str, const char* delim);
```

Splits a string into tokens using delimiter characters.

```
0 #include <stdio.h>
1 #include <string.h>
2
3 int main(void) {
4     char s[] = "red,green,blue";
5
6     char *token = strtok(s, ",");
7
8     while (token != NULL) {
9         printf("%s\n", token);
10        token = strtok(NULL, ",");
11    }
12
13    return 0;
14 }
```

strtok modifies the original string by writing '\0' characters into it. Also notice that strtok uses a hidden internal static state. Essentially, you can pass NULL to it and it will pick up where it left off.

```
0 char s[] = "red,green,blue";
1 size_t n = sizeof s;
2
3 char *token = strtok(s, ",");
4
5 for (size_t i = 0; i < n; ++i) {
6     printf("%c", s[i]);
7     if (s[i] == '\0') {
8         fputs("__null-term__", stdout);
9     }
10 }
11
12 // red__null-term__green,blue__null-term__
```



```

0 char s[] = "red,green,blue";
1 size_t n = sizeof s;
2
3 char *token = strtok(s, ",");
4
5 while (token != NULL) {
6     token = strtok(NULL, ",");
7 }
8
9 for (size_t i = 0; i < n; ++i) {
10     printf("%c", s[i]);
11     if (s[i] == '\0') {
12         // Using fputs instead of puts for no newline.
13         fputs("\\0", stdout);
14     }
15 }
16
17 // red\0green\0blue\0

```

- **strspn:**

```

0 size_t strspn(const char* s, const char* accept);

```

Returns the length of the initial segment of *s* containing only characters from *accept*.

- **strcspn:**

```

0 size_t strcspn(const char *s, const char *reject);

```

Returns the length of the initial segment of *s* containing no characters from *reject*.

- **strpbrk:**

```

0 char *strpbrk(const char *s, const char *accept);

```

Finds the first character in *s* that matches any character in *accept*.

- **strerror:**

```

0 char* strerror(int errnum);

```

Returns a human-readable error message for an error number.

## Memory operations in string.h

- **memset**

```
0 void* memset(void* s, int c, size_t n);
```

Sets  $n$  bytes of memory to the byte value  $c$ .

Returns  $s$

```
0 char buffer[8];  
1  
2 memset(buffer, 0, sizeof buffer);  
3  
4 printf("%d\n", buffer[0]);
```

- **memcpy:**

```
0 void *memcpy(void *dest, const void *src, size_t n);
```

Copies exactly  $n$  bytes from  $src$  to  $dest$ .

Returns  $dest$ .

```
0 int src[] = {1, 2, 3};  
1 int dest[3];  
2  
3 memcpy(dest, src, sizeof src);
```

Overlapping memory is UB.

- **memmove:**

```
0 void *memmove(void *dest, const void *src, size_t n);
```

Copies exactly  $n$  bytes from  $src$  to  $dest$ , safely handling overlapping memory regions.

Returns  $dest$ .

- **memcmp:**

```
0 int memcmp(const void *s1, const void *s2, size_t n);
```

Compares exactly  $n$  bytes of memory.

Returns:

- $< 0$  first differing byte in  $s1$  is less than the one in  $s2$
- $0$  all  $n$  bytes are equal
- $> 0$  first differing byte in  $s1$  is greater than the one in  $s2$

- **memchr:**

```
0 void* memchr(const void* s, int c, size_t n);
```

Searches the first  $n$  bytes of memory for byte value  $c$ .

Returns pointer to the first matching byte or NULL.

## stdlib.h

### 5.1 Memory management

- **malloc**

```
0 void* malloc(size_t size);
```

Allocates `size` bytes of uninitialized memory.

Returns a pointer to the allocated memory, or NULL if allocation fails.

```
0 int* arr = malloc(10 * sizeof *arr);
```

The memory contains indeterminate values. Reading before writing is undefined behavior.

- **calloc**:

```
0 void* calloc(size_t n, size_t size);
```

Allocates space for  $n$  elements, each `size` bytes, and initializes all bytes to zero.

Returns a pointer to the allocated memory, or NULL if allocation fails.

```
0 int* arr = calloc(10, sizeof *arr);
```

For integer types, the values become 0. For pointer types, on normal modern systems this gives null pointers, but the standard technically describes byte zeroing.

- **realloc**:

```
0 void* realloc(void* ptr, size_t size);
```

Changes the size of a previously allocated block.

Returns a pointer to the resized block, or NULL if resizing fails.

```

0  int *arr = malloc(5 * sizeof *arr);
1
2  int *tmp = realloc(arr, 10 * sizeof *arr);
3  if (!tmp) {
4      free(arr);
5      /* allocation failed */
6  } else {
7      arr = tmp;
8  }
9
10 free(arr);

```

If you assign the return value of `realloc` to the original memory pointer, you lose it if `realloc` fails and returns `NULL`.

- **free**

```

0  void free(void* ptr);

```

Releases memory allocated by `malloc`, `calloc`, or `realloc`.

Returns nothing.

```

0  free(NULL); // safe
1  free(nullptr) // safe

```

Both of these are safe, but freeing the same pointer twice is undefined behavior.

## 5.2 String to number conversions

- **\_\_Nullable**: Simply means that the pointer passed to the function is allowed to be NULL. It is however not a keyword in C (and doesn't occur even once in the draft for the C23 standard) but an extension supported only by some compilers, for example clang and icx.
- **atoi, atol, atoll**:

```
0  int atoi(const char* nptr);
1  long atol(const char* nptr);
2  long long atoll(const char* nptr);
```

Converts a string to int, long, or long long.

Returns the converted integer.

No reliable error reporting, prefer `strtol` or `strtoll`.

- **strtol, strtoll, strtoul, strtoull, strtod, strtodf, strtold**

```
0  long strtol(const char *restrict nptr, char **restrict
   ↳ endptr, int base);
1  long long strtoll(const char *restrict nptr, char **restrict
   ↳ endptr, int base);
2  unsigned long strtoul(const char *restrict nptr, char
   ↳ **restrict endptr, int base);
3  unsigned long strtoull(const char *restrict nptr, char
   ↳ **restrict endptr, int base);
4  double strtod(const char *restrict nptr, char **restrict
   ↳ endptr);
5  float strtodf(const char *restrict nptr, char **restrict
   ↳ endptr);
6  long double strtold(const char *restrict nptr, char
   ↳ **restrict endptr);
```

Returns the converted value. If `endptr` is not NULL, it is set to the first character after the parsed number.

```
0  char *end;
1  long x = strtol("123abc", &end, 10);
2
3  printf("%ld\n", x);    /* 123 */
4  printf("%s\n", end);  /* abc */
```

Base == 0 detects prefixes.

```
0  "123"    /* decimal */
1  "077"    /* octal */
2  "0xff"   /* hexadecimal */
```

```
0  char* end;
1  const char* s = "123a";
2  int x = strtol(s, &end, 10);
3
4  if (errno == ERANGE) {
5      puts("Overflow or underflow");
6  }
7
8  else if (s == end) {
9      puts("No conversion");
10 }
11
12 else if (*end == '\0') {
13     puts("Full conversion");
14 }
15
16 else {
17     puts("Some conversion");
18 }
```

## 5.3 Program termination and process control

- **exit**

```
0 void exit(int status);
```

Terminates the program with exit code **status**.

Returns nothing.

- **abort:**

```
0 void abort(void)
```

Abnormally terminates the program.

- **atexit:**

```
0 int atexit(void (*func)(void));
```

Registers a function to be called when the program exists normally.

Returns zero on success, nonzero on failure.

Functions registered with atexit are called in reverse order of registration.

- **Normal program exit cleanup:** A normal program exit (triggered by returning from `main()` or calling `exit()`) automatically performs standard runtime and stream cleanup, while application-level resource cleanup remains the responsibility of the code.

When a program terminates normally, the C standard library (`stdlib.h`) executes the following operations automatically in order:

- **Calls Termination Functions:** Executes functions registered via the `atexit()` function in the reverse order of their registration.
- **Flushes and Closes Streams:** Flushes all unwritten buffered data to open C streams (like `stdout` or `stderr`) and closes all open files.
- **Deletes Temporary Files:** Removes any temporary files created using `tmpfile()`.
- **Returns Control:** Passes the final exit status code back to the host environment (e.g., to the operating system or parent process).

- **\_Exit:**

```
0 void _Exit(int status);
```



Terminates the program immediately. No `atexit` handlers are called and normal cleanup is not performed.

- **system**

```
0  int system(const char* string);
```

Passes a command string to the host environment's command processor.

Returns implementation-defined status information.

You can also check whether a command processor exists:

```
0  if (system(NULL)) {  
1      puts("command processor available");  
2  }
```

The return value is

- If command is NULL, then a nonzero value if a shell is available, or 0 if no shell is available.
- If a child process could not be created, or its status could not be retrieved, the return value is -1 and `errno` is set to indicate the error.
- If a shell could not be executed in the child process, then the return value is as though the child shell terminated by calling `_exit(2)` with the status 127.
- If all system calls succeed, then the return value is the termination status of the child shell used to execute command. (The termination status of a shell is the termination status of the last command it executes.)

- **getenv:**

```
0  char* getenv(const char* name);
```

Gets the value of an environment variable.

Returns a pointer to the value string, or NULL if the variable does not exist.

Do not modify the returned string unless your platform specifically allows it. Treat it as read-only.

## 5.4 Random numbers

- **rand**

```
0  int rand(void);
```

Returns a pseudo-random integer in the range 0 to  $RAND_{MAX}$ .

- **srand**

```
0  void srand(unsigned int seed);
```

seeds the pseudo-random number generator used by `rand`.

If you do not call `srand`, the program behaves as though it were seeded with 1.

## 5.5 Sorting and searching

- **qsort**

```
0 void qsort(void *base, size_t nmemb, size_t size, int  
  ↪ (*compare)(const void *, const void *));
```

Sorts array. Returns nothing

The comparison function must return:

- < 0 if first element comes before second
- 0 if elements compare equal
- > 0 if first element comes after second

```
0 #include <stdio.h>  
1 #include <stdlib.h>  
2  
3 int cmp_int(const void *a, const void *b) {  
4     const int *x = a;  
5     const int *y = b;  
6  
7     if (*x < *y) return -1;  
8     if (*x > *y) return 1;  
9     return 0;  
10 }  
11  
12 int main(void) {  
13     int arr[] = {5, 2, 9, 1, 3};  
14     size_t n = sizeof arr / sizeof arr[0];  
15  
16     qsort(arr, n, sizeof arr[0], cmp_int);  
17  
18     for (size_t i = 0; i < n; ++i) {  
19         printf("%d ", arr[i]);  
20     }  
21 }
```

- **bsearch:**

```
0 void *bsearch(const void *key, const void *base, size_t  
  ↪ nmemb, size_t size, int (*compar)(const void *, const  
  ↪ void *));
```

Performs binary search on a sorted array.

Returns pointer to the matching element, or NULL.

```

0  #include <stdio.h>
1  #include <stdlib.h>
2
3  int cmp_int(const void *a, const void *b) {
4      const int *x = a;
5      const int *y = b;
6
7      if (*x < *y) return -1;
8      if (*x > *y) return 1;
9      return 0;
10 }
11
12 int main(void) {
13     int arr[] = {1, 2, 3, 5, 9};
14     int key = 5;
15
16     int *found = bsearch(&key, arr, 5, sizeof arr[0],
17                          ↪ cmp_int);
18
19     if (found) {
20         printf("found %d\n", *found);
21     }
22 }

```

## 5.6 Integer math helpers

- ```
0  int abs(int j);
1  long labs(long j);
2  long long llabs(long long j);
3
4  div_t div(int numer, int denom);
5  ldiv_t ldiv(long numer, long denom);
6  lldiv_t lldiv(long long numer, long long denom);
```

- `div_t`:

```
0  typedef struct {
1      int quot;
2      int rem;
3  } div_t;
```

Its variants are similar.

## Concepts

- **Tag names:** In C, a tag name is the name that belongs to a struct, union, or enum declaration. For example,

```
0 struct Node {  
1     int value;  
2 };
```

Here, `Node` is the **tag name**. The actual type is `struct Node`. So, to declare a variable

```
0 struct Node n;
```

This is why we see

```
0 typedef struct Node {...} node;  
1 typedef union _U {...} U;
```

Then,

```
0 U u;  
1 Node n;
```

Are both legal. If we don't care about writing

```
0 Union _U u;  
1 Struct Node n;
```

We can omit the tag name,

```
0 typedef struct {...} Node;  
1 typedef union {...} U;
```

Then, only

```
0 Node n;  
1 U u;
```

Are legal.

- **Functions that return function pointers:** Consider a function  $f$  that accepts a single integer.

```
0  f(int x) {...}
```

Suppose  $f$  returns a pointer a function that returns void and accepts a single double.

```
0  typedef void (*fp)(double);
1  fp f(int x) {...}
```

But, what if we didn't want to use typedef. Then, we wrap the function  $f$  inside the function pointer type.

```
0  void (*f(int x))(double y) { ... }
```

Notice how  $fp$  became the signature (excluding a return value) of  $f$ . Read as: " $f$ " is a function that takes an int and returns a pointer to a function that takes a double and returns void.

- **Unions:** C code uses union more often because C has fewer built-in abstraction tools. In C++, many uses of union are replaced by safer language/library features such as classes, inheritance, templates, `std::variant`, `std::optional`, and RAII-managed types.

A union is useful when you want several different members to occupy the same memory location, but only one of them is valid at a time

```
0  union Value {
1      int i;
2      double d;
3      char* s;
4  }
```

The size of Value is large enough to hold the largest member. The fields overlap in memory.

A classic C pattern is a tagged union:

```
0  enum ValueKind {  
1      VALUE_INT,  
2      VALUE_DOUBLE,  
3      VALUE_STRING  
4  };  
5  
6  struct Value {  
7      enum ValueKind kind;  
8  
9      union {  
10         int i;  
11         double d;  
12         char* s;  
13     } as;  
14  };
```

The kind field tells you which union member is currently valid.



## 6.1 C99

- **Designated initializers:** Designated initializers allow initializing struct fields by name.

```
0 struct S {
1     int x,y,z;
2 };
3
4 struct S s = {.x = 10 };
5 struct S s2 = {.x = 10, .y = 20, .z = 30};
6
7 printf("%d, %d, %d\n", s.x, s.y, s.z); // 10, 0, 0
8 printf("%d, %d, %d\n", s2.x, s2.y, s2.z); // 10, 20, 30
```

- **Designate array indices:** You can also designate array indices:

```
0 int arr[10] = {
1     [5] = 6,
2     [2] = 1
3 };
4
5 // 0, 0, 1, 0, 0, 6, 0, 0, 0, 0,
6 for (__auto_type i = 0; i < 10; ++i)
7     printf("%d, ", arr[i]);
```

- **\_\_auto\_type:** \_\_auto\_type is a GNU C compiler extension supported by GCC and Clang that enables automatic type inference in the C programming language. It allows the compiler to automatically deduce the data type of a local variable from its initialization expression, providing functionality similar to the auto keyword in C++11
  - **Immediate Initialization Required:** You cannot declare a variable with \_\_auto\_type without assigning a value immediately.
  - **Plain Identifiers Only:** The declarator must be a single, plain variable name. You cannot mix it with pointer or array syntax during declaration (e.g., \_\_auto\_type \*ptr = x; is syntax error).
  - **Single Variable Limit:** You can only declare one variable per statement

Before \_\_auto\_type, type-generic macros relied on the typeof extension. However, typeof evaluates expressions with side effects twice. Using \_\_auto\_type safely captures values exactly once, preventing double-evaluation bugs:

- **auto:** The behavior of the auto keyword in C depends entirely on the version of the C standard you are using. In traditional C (up to C17), auto is a storage class specifier used to declare variables with automatic storage duration, which means they are created when entering a block and destroyed upon exiting it. However, starting with the C23 standard, auto has been redefined to perform type inference, matching its modern behavior in C++
- **Compound literals:** Compound literals create unnamed objects inline.

```

0  struct Point {
1      int x;
2      int y;
3  };
4
5  draw_point((struct Point){ .x = 10, .y = 20 });

```

They are also useful for arrays:

```

0  int *p = (int[]){1, 2, 3, 4};

```

Inside a function, a compound literal has automatic storage duration. At file scope, it has static storage duration.

- **Variadic macros:** C99 added macros that accept a variable number of arguments.

```

0  #define LOG(fmt, ...) \
1  fprintf(stderr, "[log] " fmt "\n", __VA_ARGS__)

```

- **String concatenation at compile time:** This is allowed because C concatenates adjacent string literals at compile time.

```

0  printf("hello, " "world!");

```

Is translated as if you had written

```

0  printf("Hello, world!");

```

The two string literals are not separate runtime strings. The compiler combines them during translation, before the program runs.

This rule exists mainly to make long strings easier to write:

```

0  printf("This is a very long message "
1  "split across multiple source lines "
2  "but still one string.\n");

```

- **Inline functions:** C99 added the inline keyword.

```

0  inline int square(int x) {
1      return x * x;
2  }

```

The purpose is to allow function-like abstractions without the problems of macros.

```
0  #define SQUARE(x) ((x) * (x))
1  SQUARE(i++) // Increments i twice
```

An inline function would avoid the issue above.

C99 inline rules are slightly subtle, especially across translation units. For header-defined functions, many projects use static inline. This gives each translation unit its own internal-linkage copy.

```
0  static inline int square(int x) {
1      return x*x;
2  }
```

- **Variable Length Arrays (VLAs):** C99 added variable length arrays, or VLAs.

```
0  void f(int n) {
1      int arr[n];
2
3      for (int i = 0; i < n; ++i) {
4          arr[i] = i;
5      }
6  }
```

The array size is determined at runtime, but the object usually has automatic storage duration, typically stack-like storage.

VLAs are controversial. They are convenient, but they can be dangerous if *n* is large or untrusted:

C11 later made VLAs optional, so they are less universally portable than other C99 features.

- **Flexible array members:** C99 added flexible array members for structs with variable-sized trailing data.

```
0  struct Buffer {
1      size_t len;
2      unsigned char data[];
3  };
```

You allocate enough memory for the struct plus the extra array data:

```
0  struct Buffer *b = malloc(sizeof *b + 1024);
1
2  b->len = 1024;
3  b->data[0] = 42;
```

The flexible array member must be the last member of the struct.

- `__func__`: C99 added `__func__`, which expands to the current function name as a string.

```
0  void parse_expr(void) {
1      printf("entered %s\n", __func__);
2  }
```